ASDD PRACTICAL EXAM

Step-by-Step Procedure for All 7 Experiments

Practical preparation guide | GitHub • Jira • Docker • Terraform • Grafana

Important: This document focuses on the practical steps from the experiment list you provided. Theory questions are listed at the end of each experiment as a reminder; their full answers are not included in this steps-only guide.

Before the practical exam: One-time setup

A. Check the required accounts and software

- GitHub account and Git installed on your computer.
- Access to Jira (use the account or project space provided by your college).
- Docker Desktop installed and running.
- A terminal such as Git Bash, PowerShell, Command Prompt or VS Code terminal.
- Internet connection.

B. Generate a GitHub Personal Access Token (PAT)

1. Open https://github.com/settings/tokens and sign in.
2. Open Tokens (classic), then select Generate new token → Generate new token (classic).
3. Enter a note such as ASDD Practical Git Push.
4. Choose an expiry period as instructed by your college.
5. Under Select scopes, select repo. This grants repository access; use the minimum permissions required.
6. Select Generate token.
7. Copy the token immediately and keep it private. GitHub will not show the full token again.

Security: Never paste your token into source code, screenshots, chat, or a public repository. If it is exposed, revoke it and create a new one.

C. Use the token when pushing

8. Open a terminal in your local Git repository.
9. Run git push.
10. When asked for a username, enter your GitHub username.
11. When asked for a password, paste the PAT (not your GitHub account password).
12. Press Enter. If Git does not prompt, your credential manager may already have saved credentials.

Tip: The token is used as the password for HTTPS Git operations. SSH authentication is another option, but follow the method specified by your instructor.

Experiment 1: Jira Board and Sprint Management

Aim: Create a Jira board, prepare Product and Sprint Backlogs, add user stories, run a sprint and move stories through the workflow.

Requirements: Jira account and access to create a project.

Procedure

13. Sign in to Jira.
14. From Projects, choose Create project.
15. Choose the Scrum template (or the template specified by your instructor).
16. Enter a project name, for example Student Management System, and create the project.
17. Open the project backlog. If Jira asks you to create a board, create a Scrum board for this project.
18. Create 5–6 user stories. Example: “As an admin, I want to add a student so that student records can be maintained.”
19. For each story, enter a clear summary and description. Add acceptance criteria if the fields are available.
20. Set priority and estimate/story points if instructed.
21. Ensure the stories are in the Product Backlog. This is the list of all work that may be done for the product.
22. Click Create sprint if a sprint does not already exist.
23. Select 5–6 stories (or the number requested by the examiner) and drag them into the Sprint Backlog.
24. Open the sprint menu and choose Start sprint.
25. Enter the sprint name, duration and sprint goal, then confirm Start.
26. Open the Active Sprint board.
27. Move each story from To Do to In Progress as work begins.
28. Move the story to In Review when development is complete and it is ready for checking. If In Review is not present, add/configure that status only if you have permission; otherwise use the available workflow statuses.
29. After review/testing, move the story to Done.
30. Open each issue and add a meaningful comment, for example “Implementation completed; ready for testing.”
31. Check that the board reflects the latest status of all stories.

What to verify

- The board is associated with the correct project.
- The Product Backlog contains the product's pending work.
- The Sprint Backlog contains the selected work for the current sprint.
- The stories move through the required statuses and comments are visible.

Theory questions on the sheet: Differentiate Product Backlog and Sprint Backlog. Define both. Explain how a sprint is carried out in an Agile team.

Experiment 2: Jira and GitHub Integration

Aim: Create a Jira story, create a GitHub branch using the story key, and connect development activity to the Jira issue.

Repository to fork: https://github.com/shruthi1722/node_test.git

Procedure

32. Sign in to Jira and open the project used for the practical.
33. Create a new Story. Give it a meaningful summary and description.
34. Record the Jira issue key shown on the issue, for example ABC-12. Your actual key will be different.
35. Set the story status to To Do.
36. Open the repository URL in GitHub.
37. Click Fork and create a fork under your own GitHub account if you have not already forked it.
38. Open your fork and copy its HTTPS URL. Clone your fork, not the original repository.

```
git clone https://github.com/YOUR-USERNAME/node_test.git
cd node_test
```

39. Create a branch whose name includes the Jira issue key, for example feature/ABC-12-student-records.
40. Create and switch to the branch.

```
git checkout -b feature/ABC-12-student-records
```

41. Make a small relevant change to an existing file or add a new file.
42. Stage and commit the change. Include the Jira issue key in the commit message.

```
git add .
git commit -m "ABC-12: update student records feature"
```

43. Push the branch to your fork.

```
git push -u origin feature/ABC-12-student-records
```

44. In GitHub, open your fork and confirm the branch and commit are visible.
45. If your college has configured the Jira–GitHub integration, open the Jira issue and check its Development section for the linked branch/commit.
46. If the integration is not configured, follow the instructor's GitHub for Jira app / DVCS integration steps: an authorized Jira administrator connects the GitHub account or organization and grants repository access.
47. Optionally create a pull request from your branch to the fork's main branch, or to the original repository's main branch if the practical specifically asks for it.
48. Add the Jira issue key to the pull request title or description so the work can be associated with the issue.
49. Return to Jira and verify that development information is shown, if integration permissions and configuration allow it.

Important: A branch name containing a Jira key does not, by itself, guarantee Jira will display development details. The GitHub for Jira integration must be installed and authorized, and the repository must be connected.

Theory question on the sheet: How can Jira issues be linked to GitHub branches and commits, and how does this help others track changes?

Experiment 3: Git Commands and Version Control

Aim: Clone the forked repository, create and list branches, modify files, stage, commit and push changes.

Repository to fork: https://github.com/shruthi1722/node_test.git

Procedure

50. Open the repository URL in GitHub.
51. Click Fork and create a copy under your GitHub account if you have not already done so.
52. Open your fork and copy its HTTPS URL.
53. Open Git Bash or the VS Code terminal and clone your fork.

```
git clone https://github.com/YOUR-USERNAME/node_test.git
cd node_test
```

54. Check the current branch and working tree.

```
git status
git branch
```

55. Create a new branch and switch to it. Replace branch_name with a meaningful name.

```
git checkout -b branch_name
```

56. List branches. The asterisk (*) identifies the current branch.

```
git branch
```

57. Create a new file or edit an existing file in the repository.
58. Check which files have changed.

```
git status
```

59. Stage the changes.

```
git add .
```

60. Check status again. The staged files should appear under Changes to be committed.

```
git status
```

61. Commit the staged changes with a descriptive message.

```
git commit -m "Add practical changes"
```

62. Push the new branch to GitHub.

```
git push -u origin branch_name
```

63. Refresh your GitHub fork and open the branch selector to confirm the branch and commit are present.

If asked to push an existing branch later: You can usually use git push after its upstream has been set by the first git push -u command.

Theory questions on the sheet: Explain git add, git commit, git push, git branch and git status. Explain GitHub Actions.

Experiment 4: Docker Image Management and Container Execution

Aim: Download a Docker image, list images, run a container and verify that it is running.

Requirements: Docker Desktop installed and running.

Procedure

64. Start Docker Desktop and wait until the Docker Engine is running.
65. Open a terminal.
66. Check that Docker is available.

```
docker --version
docker info
```

67. Pull a small image from Docker Hub. For example, pull Ubuntu.

```
docker pull ubuntu
```

68. List the images downloaded on your machine.

```
docker images
```

69. Run an interactive container. The corrected command places the image name before the command to run inside it.

```
docker run -it --name myubuntu ubuntu bash
```

70. Inside the container, run a basic command to confirm that the shell is working.

```
cat /etc/os-release
exit
```

71. List running containers. The container may have stopped after you typed exit.

```
docker ps
```

72. List all containers, including stopped containers.

```
docker ps -a
```

73. To demonstrate a container that remains running, start an Nginx container in detached mode and map a host port.

```
docker run -d --name mynginx -p 8080:80 nginx
```

74. Check that the Nginx container is running.

```
docker ps
```

75. Open http://localhost:8080 in a browser. The Nginx welcome page should appear.
76. Stop and remove the test container when finished.

```
docker stop mynginx
docker rm mynginx
```

Command correction: The sheet shows `docker run -it bash image_name`. The usual correct order is `docker run -it image_name bash`, for example `docker run -it ubuntu bash`.

Theory questions on the sheet: Differentiate a Docker image and container. Explain the Docker build command.

Experiment 5: Dockerfile Creation and Image Building

Aim: Write a Dockerfile, build an image, run a container and verify the application.

Example: A simple Node.js application. If the examiner gives a different application, adjust the files and commands accordingly.

Procedure

77. Start Docker Desktop.
78. Create a folder named docker-demo and open it in VS Code.
79. Create a file named package.json.

```
{
  "name": "docker-demo",
  "version": "1.0.0",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.21.2"
  }
}
```

80. Create app.js with a simple web server.

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from Docker!');
});

app.listen(3000, '0.0.0.0', () => {
  console.log('App running on port 3000');
});
```

81. Create a file named Dockerfile (no file extension).
82. Write the Dockerfile instructions.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

83. Save all files. Build the Docker image from the folder containing the Dockerfile.

```
docker build -t my-node-app .
```

84. Check that the image was created.

```
docker images
```

85. Run a container and map host port 3000 to container port 3000.

```
docker run -d --name my-node-container -p 3000:3000 my-node-app
```

86. Check that the container is running.

```
docker ps
```

87. Open http://localhost:3000 in a browser. Confirm that “Hello from Docker!” appears.
88. If the application does not open, inspect its logs.

```
docker logs my-node-container
```

89. Stop and remove the container after the demonstration.

```
docker stop my-node-container
docker rm my-node-container
```

Remember: The dot in `docker build -t my-node-app .` means the current directory is the build context. The Dockerfile must be in that directory unless you specify another path.

Theory question on the sheet: What is a Dockerfile? Explain FROM, COPY, RUN, EXPOSE and CMD.

Experiment 6: Terraform and GitHub Actions

Aim: Fork and clone the Terraform project, edit main.tf, push the change and verify the GitHub Actions workflow.

Repository to fork: https://github.com/shruthi1722/terraform23

Procedure

90. Sign in to GitHub.
91. Open https://github.com/shruthi1722/terraform23.
92. Click Fork and create a fork under your GitHub account if you have not already done so.
93. Open your fork and copy its HTTPS URL.
94. Open a terminal and clone your fork.

```
git clone https://github.com/YOUR-USERNAME/terraform23.git
cd terraform23
```

95. List the project files and locate main.tf.

```
dir
```

96. Open the project folder in VS Code.

```
code .
```

97. Open main.tf and inspect the existing Terraform configuration.
98. Make a small valid change requested by the examiner. For example, update a resource tag or a variable value. Do not change resource types or cloud settings without understanding the effect.
99. Save main.tf.
100. Check the changed files.

```
git status
```

101. Stage and commit the change.

```
git add main.tf
git commit -m "Update Terraform configuration"
```

102. Push the change to your fork's current branch.

```
git push
```

103. Open your fork on GitHub and verify that the latest commit and main.tf change are visible.
104. Open the Actions tab in your fork.
105. Select the workflow configured in the repository.
106. Open the latest workflow run triggered by your push.
107. Check each job and step. A green tick indicates success; a red cross indicates failure.
108. Open a failed step's logs if the workflow fails and read the error message.

Important: A workflow may not run if Actions are disabled, the workflow file is absent, the trigger does not match the branch, or required repository secrets are missing. Do not run `terraform apply` against real cloud infrastructure unless the examiner specifically asks and the account is authorized.

Theory question on the sheet: What is Terraform? List Terraform commands and explain their purpose.

Experiment 7: Grafana Using Docker

Aim: Pull and run Grafana in Docker, map its internal port 3000 to localhost port 3001, and open the dashboard.

Requirements: Docker Desktop and a web browser.

Procedure

109. Start Docker Desktop and wait until the Docker Engine is running.
110. Open a terminal.
111. Pull the official Grafana image.

```
docker pull grafana/grafana
```

112. Confirm that the image is available.

```
docker images
```

113. Run Grafana in detached mode. Map host port 3001 to Grafana's container port 3000.

```
docker run -d --name grafana -p 3001:3000 grafana/grafana
```

114. Check that the Grafana container is running.

```
docker ps
```

115. Open the browser and visit http://localhost:3001.
116. At the first login, use the default credentials for the image: username admin and password admin, unless the image/version or your instructor has changed them.
117. Grafana may ask you to change the default password. Set a new password for the session.
118. Confirm that the Grafana home page/dashboard opens.
119. If the page does not load, inspect the container logs.

```
docker logs grafana
```

120. When finished, stop and remove the container if it is only a temporary practical instance.

```
docker stop grafana
docker rm grafana
```

Port mapping: `-p 3001:3000` means host port 3001 forwards to container port 3000. Therefore, use `http://localhost:3001` in the browser.

Theory question on the sheet: What is Grafana, and how is it used for monitoring and visualization in DevOps?

Quick Troubleshooting Guide

Problem	What to check
git: command not found / not recognized	Install Git, reopen the terminal and check `git --version`.
Authentication failed during git push	Use your GitHub username and PAT as the HTTPS password, or check the credential manager. Never use your normal GitHub password.
Permission denied / cannot push	Confirm that you cloned your own fork and that you have write access to it.
Docker daemon is not running	Start Docker Desktop and wait until the Engine is running; then retry.
Port is already allocated	Choose another free host port, for example `-p 3002:3000`, and open the matching localhost port.
Container exits immediately	Use `docker ps -a` and `docker logs CONTAINER_NAME` to inspect why it stopped. Interactive shells exit when you type `exit`.
Grafana page does not open	Check `docker ps`, confirm `0.0.0.0:3001->3000/tcp`, and inspect `docker logs grafana`.
GitHub Actions workflow is not visible	Check that `.github/workflows/` contains a valid `.yml` or `.yaml` file and that Actions are enabled.
Jira does not show GitHub development details	Check whether GitHub for Jira is installed, authorized and connected to the correct repository; a branch name alone is not enough.